

Concevoir et Déployer un Système **RAG** Intelligent

Assistant de recommandation d'événements culturels Puls-Events

Anna HARBA

Technologies Clés

LangChain • Mistral • FAISS •
FastAPI

Aujourd'hui je vais vous présenter mon projet de conception et déploiement d'un système RAG intelligent pour la recommandation d'événements culturels.

L'objectif était de développer un assistant conversationnel capable de répondre à des questions utilisateurs en s'appuyant sur des données réelles provenant d'OpenAgenda.

Pour ce projet, j'ai utilisé LangChain, FAISS, Mistral AI, FastAPI et Docker afin de construire un pipeline complet et industrialisable.

Contexte Métier & Problématique

Puls-Events souhaitait transformer des fiches d'événements statiques en une expérience conversationnelle intelligente.

Le principal défi était de traiter des données non structurées issues d'OpenAgenda tout en évitant les hallucinations des LLM.

Il fallait également garantir la fiabilité des informations comme les dates et les lieux, et proposer une solution facilement déployable.

La Vision Puls-Events

Développer un chatbot intelligent capable de recommander des événements culturels personnalisés à partir de données réelles.

Objectif : Transformer des fiches d'événements statiques en une expérience conversationnelle fluide.

Le Défi Technique

Comment traiter des milliers de données non structurées (OpenAgenda) et fournir des réponses fiables ?

- Éviter les **hallucinations** des LLM.
- Garantir la **véracité** des dates et lieux.
- Assurer un déploiement **industrialisable**.

Solution : Le Système RAG

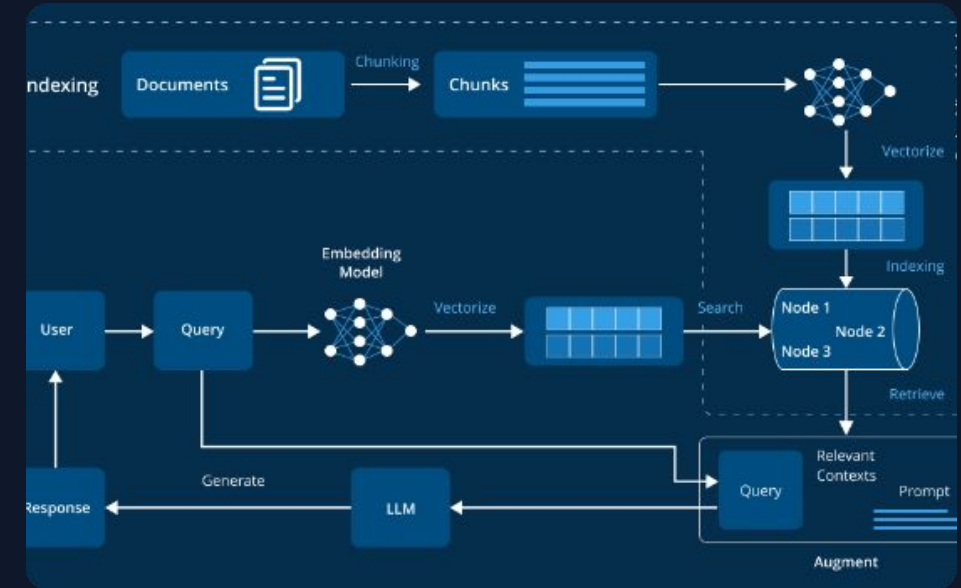
Le **Retrieval-Augmented Generation** combine le meilleur de deux mondes :

Recherche Sémantique

Identification des événements pertinents dans une base vectorielle (FAISS).

Génération Contextuelle

Utilisation de Mistral AI pour formuler une réponse basée *uniquement* sur les documents extraits.



POURQUOI Parce que les systèmes RAG sont aujourd'hui très utilisés pour intégrer les LLM dans des environnements métiers avec des données dynamiques.

Pourquoi les hallucinations sont un problème ?

Réponse :

Parce qu'un LLM peut inventer des événements ou des dates inexistantes si on ne le contraint pas avec un contexte fiable.

Pourquoi utiliser des données OpenAgenda ?

Réponse :

Parce que ce sont des données réelles, variées et régulièrement mises à jour.

Pour répondre à ce problème, j'ai développé un système RAG, c'est-à-dire Retrieval-Augmented Generation.

Le système fonctionne en deux étapes :

D'abord une recherche sémantique dans une base vectorielle FAISS afin de récupérer les événements les plus pertinents.

Ensuite, le modèle Mistral génère une réponse uniquement à partir du contexte récupéré.

Nettoyage des données

J'ai commencé par récupérer les données depuis OpenAgenda puis les nettoyer avec un script Python.

Cette étape permettait :

- de supprimer les données inutiles ;
- nettoyer le HTML ;
- structurer les événements en JSON ;
- préparer les données pour l'indexation vectorielle.

The screenshot shows a VS Code editor with a project named 'RAG-OPENAGENDA'. The Explorer sidebar on the left shows the file structure, with 'src/data_loader.py' selected. The main editor window displays the code for 'data_loader.py', which imports 'requests' and 'json', defines a URL for the OpenAgenda API, sets search parameters for 'evenements-publics-openagenda' in French for 'Paris', and saves the results to 'data/events.json'. The terminal at the bottom shows a successful POST request to the API.

```
src > data_loader.py > ...
1 import requests
2 import json
3
4 url = "https://public.opendatasoft.com/api/records/1.0/search/"
5
6 params = {
7     "dataset": "evenements-publics-openagenda",
8     "rows": 100,
9     "lang": "fr",
10    "q": "Paris"
11 }
12
13 response = requests.get(url, params=params)
14 data = response.json()
15
16 with open("data/events.json", "w", encoding="utf-8") as f:
17     json.dump(data, f, ensure_ascii=False, indent=2)
18
19 print("Data saved ✅")
```

Terminal output:

```
(env) ...
INFO: 127.0.0.1:52762 - "POST /ask HTTP/1.1" 200 OK
```

python src/clean_data.py

The screenshot shows a terminal window with the command 'python src/clean_data.py' being executed. The output shows 'Cleaned data saved' followed by a green checkmark icon.

```
(env) PS C:\Users\anna\Documents\rag-openagenda> python src/clean_data.py
Cleaned data saved ✅
(env) PS C:\Users\anna\Documents\rag-openagenda>
```

Collecte & Préparation des Données

Source : OpenAgenda

Extraction via API des événements (titre, description, date, lieu).

Nettoyage (ETL) :

- Filtrage par ville et période (< 1 an).
- Suppression des balises HTML.
- Structuration JSON pour l'indexation.

Exemple JSON Nettoyé

```
{  
  "title": "Jazz au Parc",  
  "city": "Paris",  
  "description": "Concert gratuit...",  
  "firstdate": "2024-06-15"  
}
```

Les données ont été extraites via l'API OpenAgenda.

J'ai filtré les événements :

- par ville ;
- par période ;
- et supprimé les balises HTML.

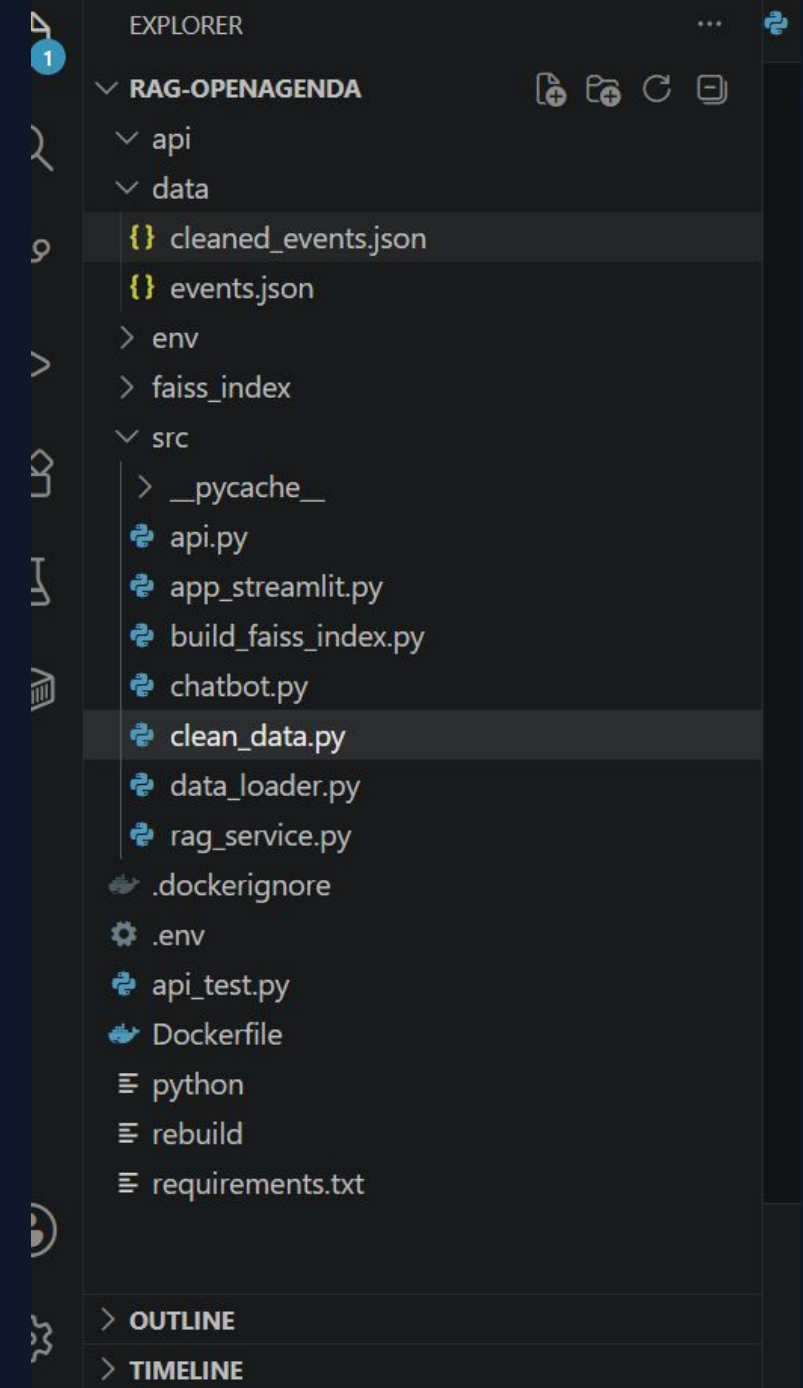
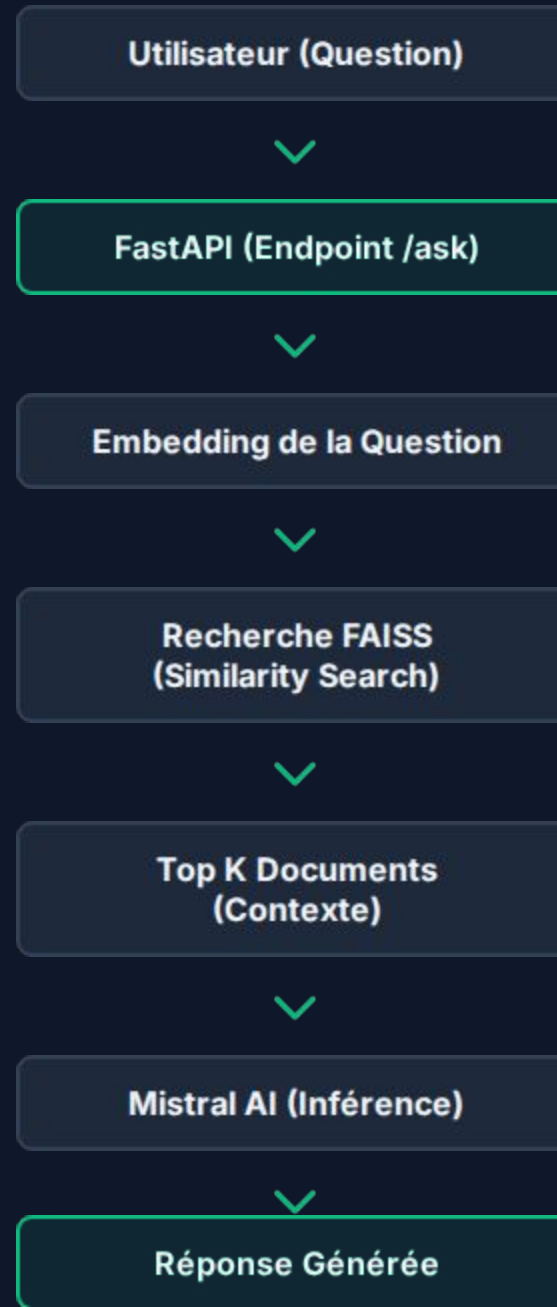
Ensuite, les événements ont été structurés dans un format JSON propre pour être indexés dans FAISS.

Architecture : Pipeline de Données

Cette slide montre le pipeline complet du système.

Lorsqu'un utilisateur pose une question :

1. La question passe par FastAPI.
2. Elle est transformée en embedding.
3. FAISS recherche les documents les plus proches.
4. Les Top-K documents sont récupérés.
5. Le contexte est envoyé à Mistral.
6. Mistral génère une réponse contextualisée.



Vectorisation & Indexation FAISS

Pour la vectorisation, j'ai utilisé Sentence-Transformers avec le modèle all-MiniLM-L6-v2.

Ce modèle transforme les textes en vecteurs de 384 dimensions.

Ensuite, ces embeddings sont stockés dans FAISS afin d'effectuer des recherches de similarité très rapides.

Moteur de Recherche Sémantique

Utilisation de **Sentence-Transformers** (all-MiniLM-L6-v2) pour transformer le texte en vecteurs.

Base FAISS (Facebook AI) :

- Recherche par similarité cosinus ultra-rapide.
- Persistance locale de l'index pour la portabilité.
- Accélération de la récupération des "Top K" résultats.

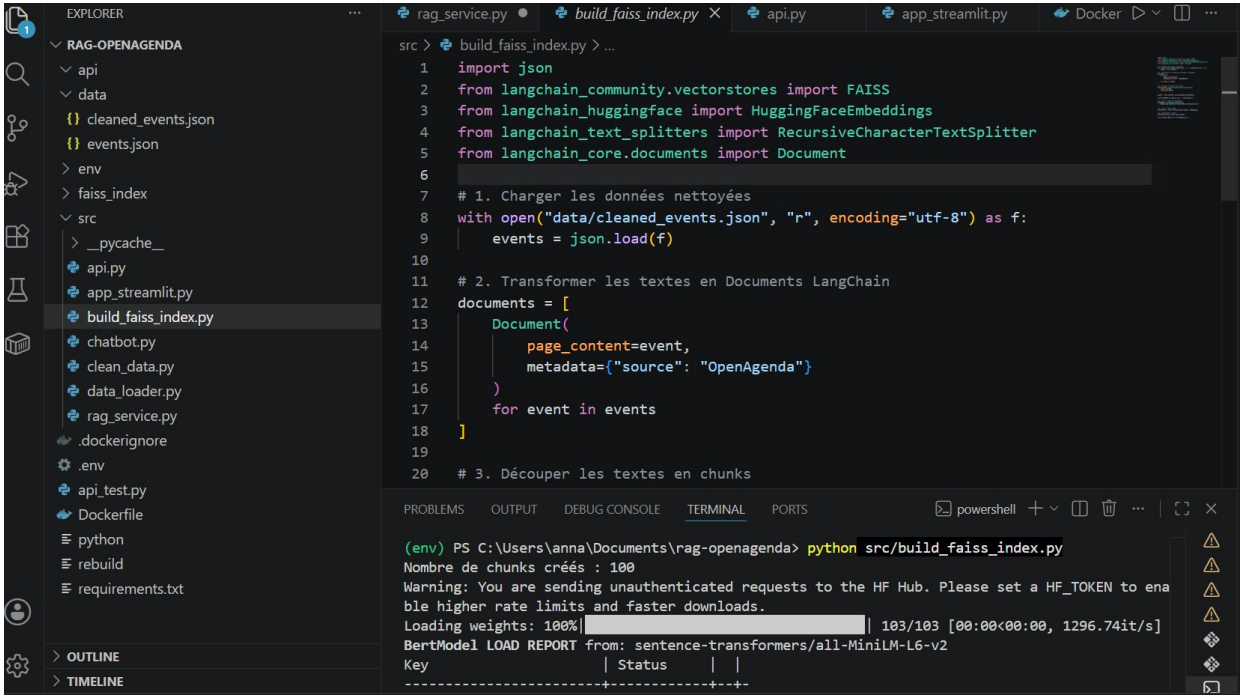
python src/build_faiss_index.py

Ici, on voit la construction réelle de l'index FAISS.

Les embeddings des événements sont générés puis sauvegardés localement.

Cela permet :

- la persistance ;
- la portabilité ;
- et des recherches rapides pendant

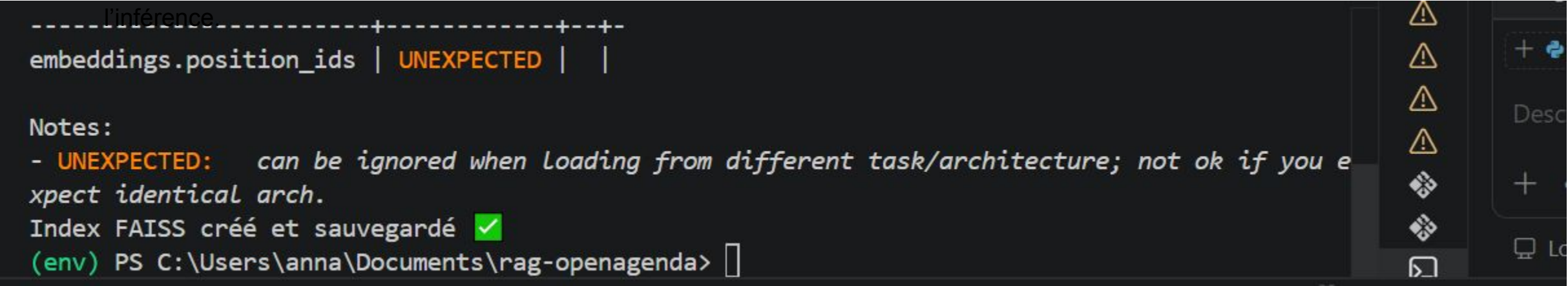


The screenshot shows a Visual Studio Code editor with a file explorer on the left and a code editor on the right. The file explorer shows a project named 'RAG-OPENAGENDA' with a 'src' folder containing 'build_faiss_index.py'. The code editor shows the content of 'build_faiss_index.py', which imports necessary libraries and defines a function to build a FAISS index from JSON data. The terminal at the bottom shows the command 'python src/build_faiss_index.py' being executed, with output indicating that 100 chunks were created and the index was successfully built.

```
src > build_faiss_index.py > ...
1 import json
2 from langchain_community.vectorstores import FAISS
3 from langchain_huggingface import HuggingFaceEmbeddings
4 from langchain_text_splitters import RecursiveCharacterTextSplitter
5 from langchain_core.documents import Document
6
7 # 1. Charger les données nettoyées
8 with open("data/cleaned_events.json", "r", encoding="utf-8") as f:
9     events = json.load(f)
10
11 # 2. Transformer les textes en Documents LangChain
12 documents = [
13     Document(
14         page_content=event,
15         metadata={"source": "OpenAgenda"}
16     )
17     for event in events
18 ]
19
20 # 3. Découper les textes en chunks
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

(env) PS C:\Users\anna\Documents\rag-openagenda> python src/build_faiss_index.py
Nombre de chunks créés : 100
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
Loading weights: 100%| 103/103 [00:00<00:00, 1296.74it/s]
BertModel LOAD REPORT from: sentence-transformers/all-MiniLM-L6-v2
Key | Status | |



The screenshot shows a terminal window with a dark background. The output of the script is displayed, including a warning about unauthenticated requests to the Hugging Face Hub and a successful completion message. The terminal prompt shows the current directory is 'C:\Users\anna\Documents\rag-openagenda'.

```
-----l'inférence-----+-----+-----+
embeddings.position_ids | UNEXPECTED | |

Notes:
- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.
Index FAISS créé et sauvegardé ✓
(env) PS C:\Users\anna\Documents\rag-openagenda>
```

• (env)

anna@DESKTOP-7HHK5HE MINGW64 ~/Documents/rag-openagenda

• \$ python api_test.py

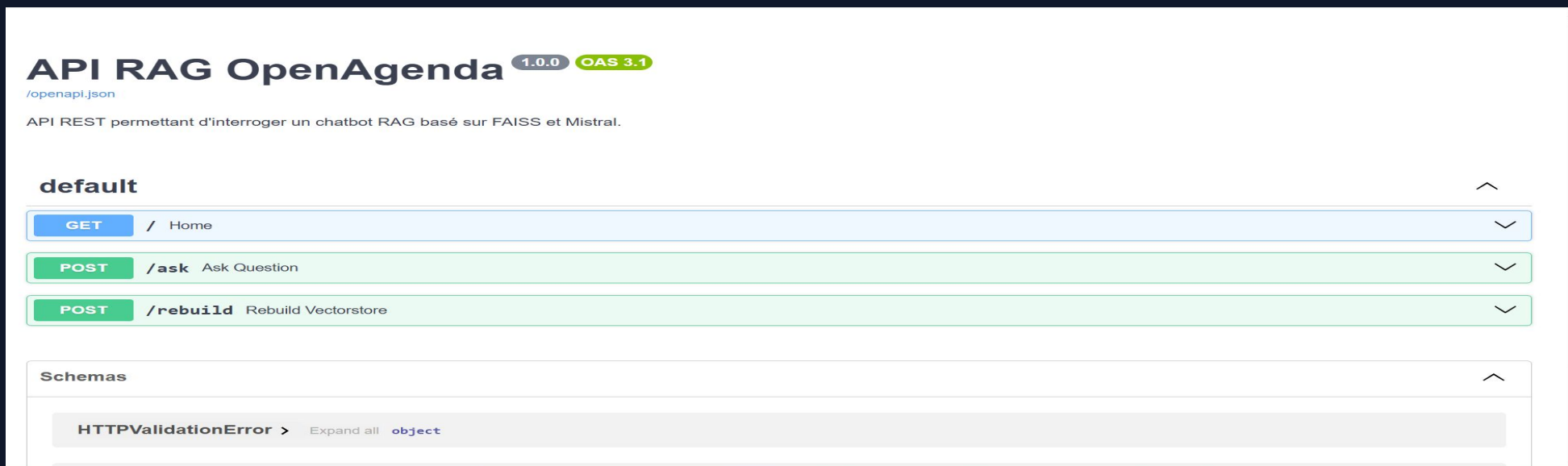
Status code: 200

Réponse JSON:

```
{'question': 'Quels événements sont disponibles à Paris ?', 'answer': "Les événements disponibles à Paris sont :\n\n1. **Paris l'été - RDV cour Lycée Jacques Decour**\n   - Date : 21 juillet 2023 à 18h30\n\n2. **Paris l'été - RDV Gare Montparnasse**\n   - Date : 21 juillet 2023 à 14h00\n\n3. **Paris Ateliers**\n   - Date : 20 septembre 2025 à 12h30\n\n4. **La petite maison dans Paris**\n   - Date : 16 octobre 2025 à 12h00"}
```

Industrialisation : FastAPI & Docker

Le système est encapsulé dans une API REST moderne pour une intégration facile.



Cas d'Usage : " Paris"

```
streamlit run src/app_streamlit.py
```

Question

"Quels concerts jazz gratuits à Paris ce week-end ?"



Récupération FAISS

Doc 1 : Jazz au Parc (Gratuit)
Doc 2 : Session Bleue (Payant)



Réponse Mistral

"Vous pouvez profiter du concert 'Jazz au Parc' samedi soir. C'est le seul événement gratuit."

```
anna@DESKTOP-7HHK5HE MINGW64 ~/Documents/rag-openagenda
$ streamlit run src/app_streamlit.py
2026-05-09 15:27:51.028 Uvicorn server started on 0.0.0.0:8501
```

You can now view your Streamlit app in your browser.

Local URL: <http://localhost:8501>

Network URL: <http://10.141.254.253:8501>

Traceback (most recent call last)

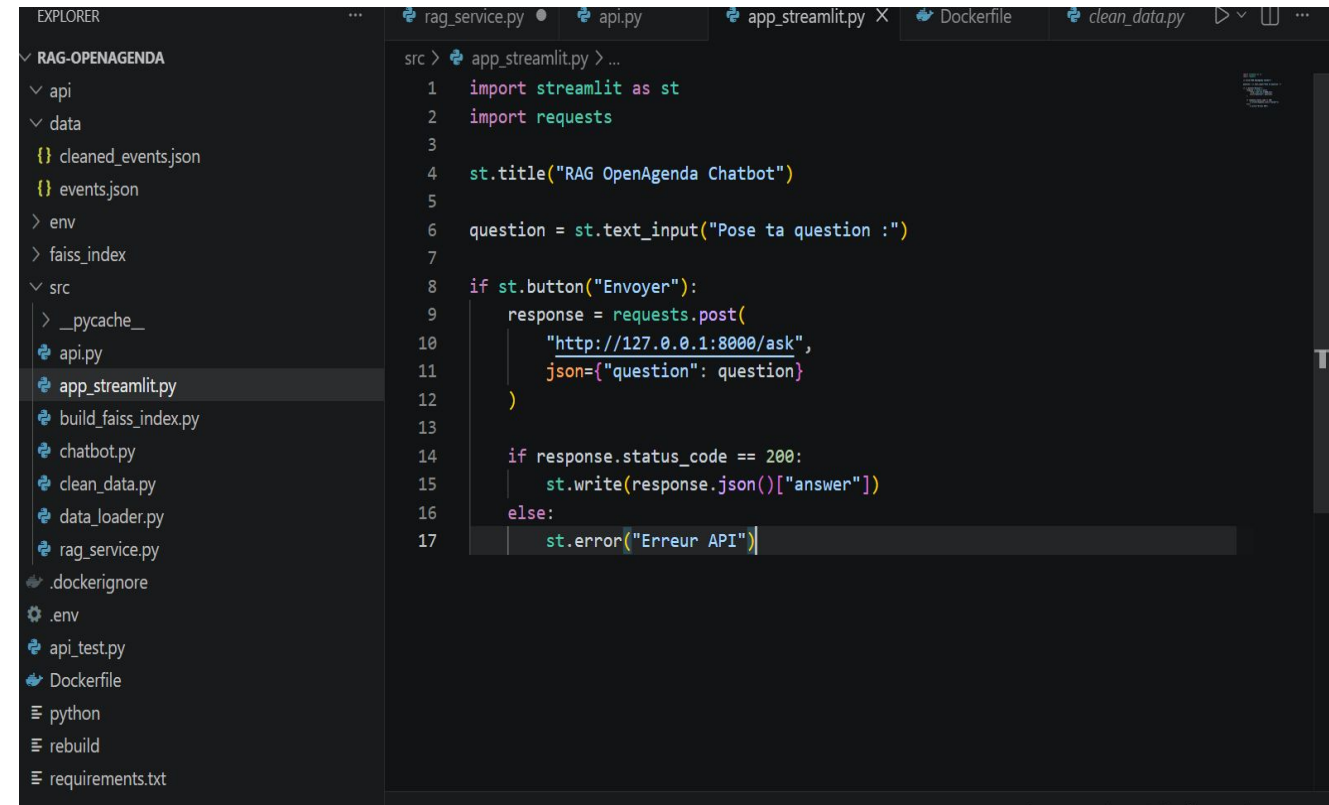
```
C:\Users\anna\Documents\rag-openagenda\env\Lib\site-packages\urllib3\connection.py:2
    in new_conn
```


RAG OpenAgenda Chatbot

Pose ta question :

Envoyer

J'ai également développé une interface Streamlit pour permettre aux utilisateurs métier de tester facilement le chatbot sans utiliser directement l'API.



The screenshot shows a VS Code editor with the Explorer sidebar on the left and the editor window on the right. The Explorer sidebar displays the project structure for 'RAG-OPENAGENDA', including folders like 'api', 'data', 'env', 'faiss_index', 'src', and files like 'api.py', 'app_streamlit.py', 'build_faiss_index.py', 'chatbot.py', 'clean_data.py', 'data_loader.py', 'rag_service.py', '.dockerignore', '.env', 'api_test.py', 'Dockerfile', 'python', 'rebuild', and 'requirements.txt'. The editor window shows the code for 'app_streamlit.py' in the 'src' directory. The code imports 'streamlit as st' and 'requests'. It sets the title to 'RAG OpenAgenda Chatbot' and creates a text input for the question. A button labeled 'Envoyer' triggers a POST request to 'http://127.0.0.1:8000/ask' with the question in the JSON body. If the response status code is 200, the answer is written to the stream; otherwise, an error is displayed.

```
EXPLORER
RAG-OPENAGENDA
├── api
├── data
│   ├── cleaned_events.json
│   └── events.json
├── env
├── faiss_index
├── src
│   ├── __pycache__
│   ├── api.py
│   ├── app_streamlit.py
│   ├── build_faiss_index.py
│   ├── chatbot.py
│   ├── clean_data.py
│   ├── data_loader.py
│   ├── rag_service.py
│   ├── .dockerignore
│   ├── .env
│   ├── api_test.py
│   ├── Dockerfile
│   ├── python
│   ├── rebuild
│   └── requirements.txt
└── ...

src > app_streamlit.py > ...
1  import streamlit as st
2  import requests
3
4  st.title("RAG OpenAgenda Chatbot")
5
6  question = st.text_input("Pose ta question :")
7
8  if st.button("Envoyer"):
9      response = requests.post(
10         "http://127.0.0.1:8000/ask",
11         json={"question": question}
12     )
13
14     if response.status_code == 200:
15         st.write(response.json()["answer"])
16     else:
17         st.error("Erreur API")
```

RAG OpenAgenda Chatbot

Pose ta question :

Quels événements sont disponibles ?

Envoyer

Voici les événements disponibles dans le contexte fourni :

1. Événement en ligne

- **Date** : 2020-01-06
- **Lieu** : Les Jardins numériques (uniquement en ligne 24h/24 et 7j/7)
- **Ville** : Non spécifiée

2. Événement

- **Date** : 2019-11-19
- **Lieu** : L'espace Icare
- **Ville** : Non spécifiée

3. Événement

Difficultés & Debugging

Gestion des versions LangChain : Incompatibilités fréquentes entre bibliothèques. **Solution** : Fixation stricte des versions dans `requirements.txt`.

Hallucinations de Dates : Le LLM mélangeait parfois les années. **Solution** : Ajout de la date actuelle dans le prompt système pour cadrer le LLM.

Nettoyage OpenAgenda : Données HTML brutes inexploitable pour FAISS. **Solution** : Pipeline de nettoyage Regex et BeautifulSoup robuste.




```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
(env)
anna@DESKTOP-7HHK5HE MINGW64 ~/Documents/rag-openagenda
$ docker run -p 8000:8000 --env-file .env rag-openagenda
ble higher rate limits and faster downloads.
Loading weights: 100%|██████████| 103/103 [00:00<00:00, 704.11it/s]
BertModel LOAD REPORT from: sentence-transformers/all-MiniLM-L6-v2
Key | Status |
-----+-----+
embeddings.position_ids | UNEXPECTED |

Notes:
- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.
INFO: Started server process [1]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO: Shutting down
INFO: Waiting for application shutdown.
INFO: Application shutdown complete.
INFO: Finished server process [1]
```

docker run -p 8000:8000 --env-file .env rag-openagenda

```
docker run -p 8000:8000 --env-file .env rag-openagenda
```

API RAG OpenAgenda 1.0.0 OAS 3.1

[/openapi.json](#)

API REST permettant d'interroger un chatbot RAG basé sur FAISS et Mistral.

default

GET / Home

POST /ask Ask Question

POST /rebuild Rebuild Vectorstore

Schemas

HTTPValidationError > Expand all [object](#)

```
{  
  "question": "Quels événements gratuits sont disponibles à Paris ?"  
}
```

Conclusion

Pour conclure :

Ce projet m'a permis de développer un pipeline RAG complet :

- collecte ;
- vectorisation ;
- génération ;
- API ;
- industrialisation.



POC RAG
Opérationnel



API REST
Industrialisée

Le POC est aujourd'hui opérationnel et peut être amélioré avec :

- mémoire conversationnelle ;
- déploiement cloud ;
- monitoring ;
- évaluation automatique.

Anna | Projet Puls-Events | Mai 2024